

C2.5 – Installation et exploitation

Guide pour installer, configurer et exploiter **CareerAI** en local, Docker et production.

1. Prérequis

Outil	Version
Node.js	18+ (20 recommandé)
npm	9+
Compte Supabase	Projet avec Auth + SQL Editor
Clé Groq	console.groq.com
Resend (optionnel)	Emails quiz / campagnes / auth SMTP

2. Installation locale

2.1 Cloner et installer

```
```bash
git clone
https://github.com/ARTHURNJOUONANG/Projet_Chat_board.git
cd Projet_Chat_board
npm install
```
```

2.2 Variables d'environnement

```
```bash
cp .env.example .env.local
```
```

Variables **obligatoires** pour un fonctionnement minimal :

| Variable | Description |
|---------------------------------|----------------------------------|
| `NEXT_PUBLIC_SUPABASE_URL` | URL projet Supabase |
| `NEXT_PUBLIC_SUPABASE_ANON_KEY` | Clé anon (publique) |
| `SUPABASE_SERVICE_ROLE_KEY` | Clé service (serveur uniquement) |
| `GROQ_API_KEY` | API IA |

Variables **recommandées** :

| Variable | Description |
|---------------------------------|-------------------------------------|
| `NEXT_PUBLIC_BASE_URL` | `http://localhost:3000` en local |
| `RESEND_API_KEY` + `EMAIL_FROM` | Emails (quiz, campagnes) |
| `CRON_SECRET` | Sécuriser `/api/cron/run-campaigns` |

Voir [.env.example](../.env.example) pour Adzuna, LBA, Puppeteer, Apollo, etc.

2.3 Schéma base de données (Supabase)

Dans **SQL Editor**, exécuter dans l'ordre :

- [supabase-schema.sql](../supabase-schema.sql) – tables de base
- [supabase-schema-saas.sql](../supabase-schema-saas.sql) –

```
recruteur, candidats, quiz
3. [supabase-migrations/campaigns.sql](../supabase-migrations/campaigns.sql) – campagnes
4. [supabase-migrations/job_campaigns_add_kind_column.sql](../supabase-migrations/job_campaigns_add_kind_column.sql) – colonne `kind`
5. [supabase-migrations/campaigns_kandi.sql](../supabase-migrations/campaigns_kandi.sql) – contacts Kandi
6. [supabase-storage-policies.sql](../supabase-storage-policies.sql) – bucket `documents`
```

Autres migrations selon besoins : `campaign\_runs\_and\_keep\_data.sql`,
`fix\_candidates\_cv\_document\_fk.sql`, etc.

2.4 Lancer en développement

```
```bash
npm run dev
```
```

Application : http://localhost:3000.

2.5 Build production local

```
```bash
npm run build
npm start
```
```

3. Docker (application web)

```
```bash
docker compose up -d --build
```
```

Fichier : [docker-compose.yml](../docker-compose.yml). Port ****3000****.

Variables passées au conteneur : Supabase, Groq (voir compose).

4. Exploitation production

****URL publique :**** https://www.careerai.live

4.1 Checklist déploiement

- [] `NEXT\_PUBLIC\_BASE\_URL=https://www.careerai.live`
- [] Migrations SQL appliquées sur le projet Supabase de prod
- [] Domaine Resend vérifié (`careerai.blog` / `careerai.live`) pour emails sortants
- [] SMTP Supabase Auth configuré (éviter `email rate limit exceeded` à l'inscription)
- [] `CRON\_SECRET` défini + tâche planifiée vers
`/api/cron/run-campaigns`

4.2 Cron campagnes

Endpoint : `GET` ou `POST` [app/api/cron/run-campaigns/route.js](../app/api/cron/run-campaigns/route.js)

En-tête : `Authorization: Bearer <CRON\_SECRET>`

Exemple cron-job.org ou Render Cron : une fois par jour.

4.3 Mobile Android

Voir [MOBILE-CAPACITOR.md](../MOBILE-CAPACITOR.md) – build Docker ou GitHub Actions.

5. Dépannage courant

Inscription : `email rate limit exceeded`

Cause : quota emails Supabase (SMTP par défaut).

Solutions :

1. Dashboard Supabase !' Authentication !' Provi
Confirm email (dev).
2. Configurer **Custom SMTP** (Resend) dans Pr
Authentication.
3. Créer l'utilisateur manuellement (Users !' A

Campagnes : colonne `kind` introuvable

Exécuter [supabase-migrations/job_campaigns_add_kind_column.sql](../supabase-migrations/job_campaigns_add_kind_column.sql).

Resend : domain not verified

Utiliser `onboarding@resend.dev` en test ou vérifier le domaine sur resend.com/domains.

Puppeteer en production

Nécessite Chromium et ~1 Go RAM. Variable `ENABLE\_BROWSER\_AUTOMATION=true` uniquement si l'hébergeur le supporte.

Port 3000 occupé

Next.js bascule sur 3001 automatiquement, ou arrêter l'autre processus.

6. Sauvegarde et maintenance

| Élément | Action |
|-----------------|---|
| ----- | ----- |
| Base PostgreSQL | Sauvegardes automatiques Supabase (selon plan) |
| Fichiers CV | Bucket `documents` – politiques RLS |
| Logs | Console hébergeur + Supabase Auth logs |
| Mises à jour | `npm update` + tests `npm test` avant déploiement |

7. Contacts et support

- Issues GitHub : [Projet_Chat_board/issues](https://github.com/ARTHUR RNJOUONANG/Projet_Chat_board/issues)
- Documentation technique : [docs/README.md](README.md)